

Clase 054 — Firmas digitales

Parte: 2 — **Criptografía aplicada** · Fuente: *Serious Cryptography* (Aumasson) y NIST FIPS 186-5 

Duración estimada: **100 min** · Nivel: **Avanzado**

Objetivo

Comprender cómo las firmas digitales proporcionan autenticidad, integridad y no repudio usando criptografía de clave pública: se firma con la clave privada y se verifica con la pública. El alumno estudiará RSA-PSS, ECDSA y Ed25519, entenderá por qué se firma el hash del mensaje y no el mensaje entero, y por qué un nonce mal generado en ECDSA puede revelar la clave privada (caso PlayStation 3).

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** el modelo firmar-con-privada / verificar-con-pública y qué garantiza.
2. **Diferenciar** firma digital de MAC (no repudio vs clave compartida).
3. **Firmar y verificar** con RSA-PSS, ECDSA y Ed25519 usando OpenSSL/Python.
4. **Explicar** por qué se firma el hash y los riesgos del nonce en ECDSA.
5. **Aplicar** firmas para verificar integridad de software.

Temas

#	Tema	Por qué importa
1	Firma vs MAC	No repudio con clave pública
2	Hash-then-sign	Eficiencia y seguridad
3	RSA-PSS	Firma RSA moderna
4	ECDSA y el riesgo del nonce	Fallo famoso (PS3)
5	Ed25519 (determinista)	Elimina el riesgo del nonce
6	Verificación de software	Uso real (paquetes, releases)
7	No repudio y sus límites	Legal y técnico

Definiciones y características

- **Firma digital:** valor generado con la clave privada que cualquiera verifica con la pública.
Característica: aporta no repudio, algo que el MAC no da.
- **No repudio:** el firmante no puede negar haber firmado, porque solo él posee la clave privada.
- **Hash-then-sign:** se firma el hash del mensaje; permite firmar mensajes grandes y liga la firma al contenido exacto.
- **RSA-PSS:** esquema de firma RSA probabilístico con prueba de seguridad; preferible a PKCS#1 v1.5.

- **ECDSA**: firma sobre curvas; requiere un nonce `k` único y aleatorio por firma. Repetirlo o predecirlo revela la clave privada.
- **Ed25519**: firma determinista (deriva `k` del mensaje y la clave), eliminando el riesgo del nonce; rápida y robusta.
- **Verificación de integridad de software**: firmar releases para que los usuarios comprueben autenticidad.

Herramientas y preparación

```
openssl version
gpg --version # opcional, para firma de software
pip install cryptography
```

Trabaja con tus propias claves. Firmar en nombre de otros sin autorización es fraude.

Laboratorio guiado

1. Firma y verifica con OpenSSL (RSA-PSS):

```
bash openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out priv.pem openssl rsa -
in priv.pem -pubout -out pub.pem openssl dgst -sha256 -sign priv.pem -sigopt
rsa_padding_mode:pss \ -out firma.bin documento.txt openssl dgst -sha256 -verify pub.pem -
sigopt rsa_padding_mode:pss \ -signature firma.bin documento.txt
```

2. **Ed25519 en Python** (ver clase 050): firma un documento y verifica; altera un byte del documento y comprueba que la verificación falla.
3. **Riesgo del nonce en ECDSA (concepto)**. Explica con la fórmula cómo, si dos firmas usan el mismo `k`, se despeja la clave privada. Es exactamente el fallo que rompió la firma de código de la PS3.
4. **Detecta manipulación de software**: firma un binario, publica la clave pública, y muestra que cualquier modificación invalida la firma.

Ejercicios

1. Explica la diferencia entre firma digital y HMAC en cuanto a no repudio.
2. Firma un archivo con RSA-PSS y verifica con la clave pública.
3. Investiga el fallo de nonce de ECDSA en la PlayStation 3.
4. ¿Por qué Ed25519 es determinista y por qué eso ayuda?
5. Verifica la firma GPG de un release real de software libre.
6. Explica por qué firmar el hash (no el mensaje) es seguro y eficiente.

Reto verificable

Construye un verificador de releases: firma un conjunto de artefactos con Ed25519, publica la clave pública y entrega un script que valide cada artefacto contra su firma. **Criterio de aceptación**: cualquier artefacto alterado o firma inválida se reporta como no confiable, y solo los íntegros pasan la verificación.

⚠ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Verificación siempre válida	No estás comparando con el documento correcto o ignoras el resultado
Nonce reutilizado en ECDSA	Revela la clave; usa RFC 6979 o Ed25519
Firmar con PKCS#1 v1.5 nuevo	Prefiere PSS para diseños nuevos
Confundir firmar con cifrar	Son operaciones distintas; firma con privada para autenticar
No verificar la clave pública del firmante	Un atacante puede sustituirla; áncala vía PKI o huella conocida

? Preguntas frecuentes

? **¿Firma = cifrar con la clave privada?** Es una simplificación peligrosa. Los esquemas modernos (PSS, EdDSA) no son "RSA al revés"; usa la primitiva de firma, no operaciones crudas.

? **¿Qué firma elijo hoy?** Ed25519 por defecto (rápida, sin riesgo de nonce). RSA-PSS o ECDSA cuando la compatibilidad lo exija.

? **¿La firma garantiza que el contenido es verdad?** No; garantiza quién lo firmó y que no cambió. La veracidad del contenido es otra cosa.

🔗 Referencias

- NIST FIPS 186-5 (firmas) — <https://csrc.nist.gov/publications/detail/fips/186/5/final>
- RFC 8032 (EdDSA) — <https://www.rfc-editor.org/rfc/rfc8032>
- RFC 6979 (nonce determinista para ECDSA) — <https://www.rfc-editor.org/rfc/rfc6979>
- Aumasson, *Serious Cryptography*, cap. 10–12.

➡ Siguiente clase

Clase 055 - PKI, certificados X.509 y autoridades de certificación